

Smart Contract Security Audit Report

Protocol: Alvara Protocol (ERC-7621 Basket Token Standard) **Audit Date:** March 6, 2026 **Auditor:** Raju Raj Kishore **Framework:** Hardhat | Solidity ^0.8.26 / ^0.8.20 / ^0.8.9

Table of Contents

- 1. Executive Summary
- 2. Scope
- 3. Protocol Overview
- 4. Contract Architecture
- 5. Trust Boundaries
- 6. Finding Statistics
- 7. Critical Findings
- 8. High Findings
- 9. Medium Findings
- 10. Low Findings
- 11. Informational Findings
- 12. General Recommendations
- 13. Disclaimer

1. Executive Summary

Alvara Protocol implements the ERC-7621 basket token standard, enabling creation and management of diversified on-chain token baskets represented as NFTs with LP-based liquidity, on-chain management fees, and governance via veALVA staking.

This audit covered all production contracts across the Alvara ecosystem including basket management, liquidity pairing, staking, governance, token contracts, and utility libraries. The audit employed a multi-phase methodology including static analysis, cross-contract data flow tracing, economic attack simulation, adversarial role analysis, and deep function-level line-by-line review.

124 findings were identified across all contracts.

The most severe systemic risks are:

- A hardcoded testnet fee period (1800 seconds instead of 30 days) that causes management fees to compound 1440x faster than intended, allowing basket owners to drain all LP holders within days
- A custom WETH implementation explicitly labeled as a testnet faucet contract deployed in production scope, giving the owner unconditional fund drain, mint, and burn capabilities
- The GREYLIST_MANAGER_ROLE on the ALVA token can greylist protocol system contracts, permanently locking all staked ALVA and DAO funds
- The DEFAULT_ADMIN_ROLE on veALVA can burn any user's voting power, making their unstake() call permanently revert and trapping their ALVA indefinitely
- Absence of signature binding (no chainId, msg.sender, or nonce) enabling cross-chain replay and Manager NFT hijacking

Severity	Count
HIGH	18
MEDIUM	32
LOW	52
INFORMATIONAL	22
Total	124

2. Scope

File	Lines	Description
contracts/BSKT.sol	1227	Core basket token (ERC-721 + basket management)
contracts/Factory.sol	904	Factory for deploying BSKT + BSKTPair
contracts/tokens/BSKTPair.sol	463	LP token pair, holds reserves
contracts/AlvaStakingV3.sol	1072	ALVA staking V3 (latest)

contracts/ File	997 Lines	ALVA staking V1 Description
contracts/tokens/Alvara.sol	127	ALVA ERC-20 token (Ethereum)
contracts/tokens/AlvaraAvax.sol	41	ALVA token (Avalanche)
contracts/utills/BSKTUtils.sol	120	Signature verification
contracts/AlvaraDao.sol	103	DAO staking (ALVA to xALVA)
contracts/tokens/WETH.sol	53	Custom Wrapped ETH
contracts/tokens/veALVA.sol	38	Vote-escrowed ALVA
contracts/utills/WithSupervisedTransfers.sol	81	Pre-listing transfer supervision (Ethereum)
contracts/utills/WithSupervisedTransfersAvax.sol	86	Pre-listing transfer supervision (Avalanche)
contracts/BSKTBeacon.sol	8	Beacon for BSKT upgrades
contracts/tokens/BSKTPairBeacon.sol	8	Beacon for BSKTPair upgrades
Total	~6,326	

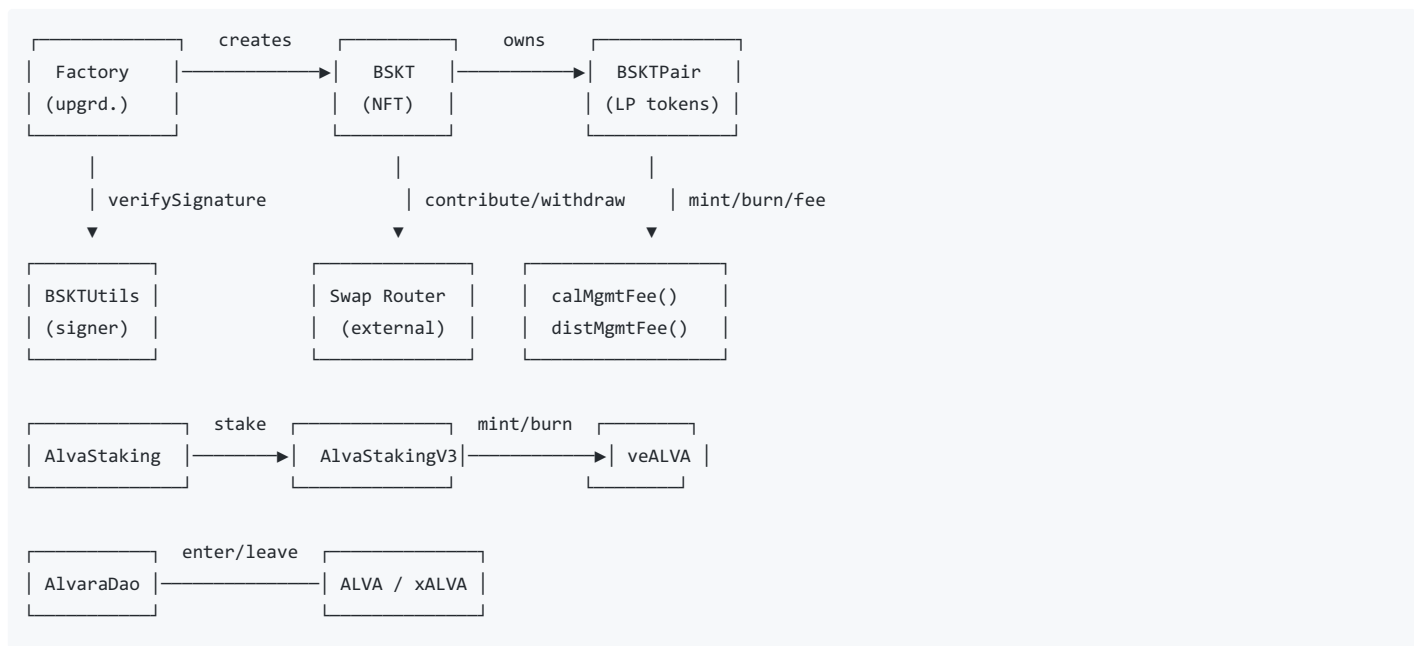
3. Protocol Overview

Alvara Protocol is an on-chain basket management protocol. Users create diversified token portfolios (baskets) represented as ERC-721 NFTs. Each basket has an associated LP pair contract (BSKTPair) that holds underlying token reserves and mints LP tokens to contributors. Basket managers earn management fees in LP tokens, claimable as ETH or individual tokens. The ALVA token provides governance through a staking system that mints non-transferable veALVA voting power.

Value Flows:

- ETH (minus fee) → 1inch Router → Tokens → BSKTPair (reserves) → LP tokens → Contributor
- LP tokens → BSKTPair.burn() → Underlying tokens or ETH → Withdrawer
- ALVA → AlvaStakingV3 (locked/burned) → veALVA (voting power) + Weekly rewards

4. Contract Architecture



5. Trust Boundaries

Component	Trust Level	Risk if Compromised
Factory UPGRADER_ROLE	Critical	Instant drain of all baskets
BSKTUtils owner	Critical	Bypass all signature verification
BSKTUtils signer (off-chain)	Critical	Authorize any swap or rebalance
veALVA DEFAULT_ADMIN_ROLE	High	Lock any user's ALVA permanently
ALVA GREYLIST_MANAGER_ROLE	High	Lock entire staking/DAO system
AlvaStakingV3 ADMIN_ROLE	High	Drain entire staking pool
AlvaraDao owner	High	Drain all DAO funds via UUPS upgrade
BSKTPair owner (BSKT contract)	Medium	Drain basket reserves via beacon upgrade
Basket Manager (NFT holder)	Medium	Malicious rebalance to worthless tokens
ALVA DEFAULT_ADMIN_ROLE	Medium	Unilateral greylist control

6. Finding Statistics

Severity	Count	% of Total
HIGH	18	14.5%
MEDIUM	32	25.8%
LOW	52	41.9%
INFORMATIONAL	22	17.7%
Total	124	

Contract	H	M	L	I	Total
Factory.sol	3	5	4	3	15
BSKT.sol	1	5	3	3	12
BSKTPair.sol	2	3	9	0	14
AlvaStakingV3.sol	2	11	17	3	33
AlvaraDao.sol	4	2	3	0	9
BSKTUtils.sol	1	3	1	1	6
veALVA.sol	1	2	2	2	7
WETH.sol	1	0	3	2	6
AlvaraAvax.sol	1	3	4	2	10
Alvara.sol	1	4	4	2	11
BSKTUtils across protocol	1	3	1	1	6

7. HIGH Severity Findings

H-01 | Custom WETH with Owner Drain, Mint and Burn Capabilities

File: contracts/tokens/WETH.sol | **Lines:** 31-44

The developer comment at lines 3-8 explicitly labels this as a testnet faucet: "This is custom WETH token smart-contract, to solve the faucet issue." Three onlyOwner functions provide unconditional fund control. withdraw() drains the entire ETH balance. mint(address, uint256) creates unlimited unbacked WETH. burn(address, uint256) destroys any user's balance without consent. Factory.weth() references this contract for LIQUIDATE-mode rebalancing. A single owner key compromise extracts all WETH-denominated basket reserves simultaneously. The withdraw(uint256) function uses .transfer() which fails for multisig recipients.

Recommendation: Replace with canonical WETH9. If a custom contract is required, remove withdraw() with no parameters, remove mint(), remove burn(), replace .transfer() with .call{value}(), and add constructor() { _disableInitializers(); }.

Severity: HIGH (CRITICAL if deployed to mainnet)

H-02 | First-Depositor xALVA Inflation Attack Steals All Subsequent Deposits

File: contracts/AlvaraDao.sol | **Lines:** 45-59

enter() calculates xALVA shares using ALVA.balanceOf(address(this)) which reads actual on-chain balance. An attacker calls enter(1) as first depositor receiving 1 xALVA. The attacker directly transfers 10,000 ALVA to the DAO. When a victim calls enter(5000 ALVA), the calculation becomes $(5000 * 1) / 10,001 = 0$. The victim's ALVA.transferFrom succeeds but they receive 0 xALVA. The attacker calls leave(1) and recovers all ALVA including the victim's deposit.

Recommendation: Mint 1000 wei to address(0xdead) on first deposit. Add require(_userShare > 0). Transfer ALVA in before minting shares (CEI compliance).

Severity: HIGH

H-03 | Lock Period Reset on Every enter() Call Traps User Funds

File: contracts/AlvaraDao.sol | **Lines:** 57, 91-95

Line 57 unconditionally sets userLockPeriod[msg.sender] = block.timestamp on every enter() call. A user with 89 days elapsed on a 90-day lock who calls enter(1 wei) to compound resets their entire clock to now. updateLockPeriod() has no upper bound — owner can set to type(uint256).max, permanently locking all funds.

Recommendation: Only set userLockPeriod on first deposit. Add require(_lockedPeriod <= 365 days) in updateLockPeriod().

Severity: HIGH

H-04 | ALVA Greylist DoS Permanently Bricks All Unstakes and DAO Withdrawals

File: contracts/tokens/Alvara.sol | **Lines:** 106-114

_transfer() reverts with ActionRestricted when the sender (from) is greylisted. If GREYLIST_MANAGER_ROLE greylists the AlvaStakingV3 contract, all ALVA.transfer() calls from staking revert — blocking every unstake() and claimRewards() permanently. Greylisting AlvaraDao blocks all leave() calls. A single transaction locks all protocol funds for all users simultaneously.

Recommendation: Add a protocol contract registry that prevents GREYLIST_MANAGER_ROLE from greylisting system addresses. Require governance approval before system contracts can be greylisted.

Severity: HIGH

H-05 | UUPS Owner Can Upgrade AlvaraDao Implementation to Drain All Funds

File: contracts/AlvaraDao.sol | **Lines:** 97-101

AlvaraDao inherits UUPSUpgradeable with _authorizeUpgrade restricted only to onlyOwner. The owner can replace the implementation with a malicious contract that transfers all staked ALVA to an attacker address in a single call. No timelock, no multi-sig, no governance.

Recommendation: Implement 48-72 hour TimelockController for upgrades. Transfer ownership to a multi-sig. Emit UpgradeProposed event giving users time to exit before upgrade executes.

Severity: HIGH

H-06 | Management Fee Period Hardcoded to 1800 Seconds (Testnet Value)

File: contracts/tokens/BSKTPair.sol | **Lines:** 259, 347

Developer comment explicitly states "1800 seconds = 30 mins for testnet; use 30 days for mainnet." On mainnet, one real month = $2,592,000 / 1800 = 1440$ contract months. With 0.5% monthly fee rate, the basket owner receives ~99.93% of all LP tokens within 30 real days. distMgmtFee() is public — any bot can call it every 30 minutes to maximize extraction. Completely dilutes all LP holders within days.

Recommendation: Replace 1800 with uint256 public constant FEE_PERIOD = 30 days. Update both calFee() and distMgmtFee() to use FEE_PERIOD.

Severity: HIGH

H-07 | LP Share Price Manipulation via Flash Loan DEX Spot Price

File: contracts/tokens/BSKTPair.sol | **Lines:** 276-285, 436-461

_totalReservedETH() values existing reserves using the new contributor's own swap execution price: (reserve * ethAmount) / tokenAmount. Flash loan to crash a basket token's price on DEX causes swap to return more tokens per ETH, driving tokenAmount high and reservedETH near zero. The LP minting formula then mints excess LP. With 10x price crash, attacker gets 100 LP instead of fair 10 LP, representing 10x dilution. Attacker burns inflated LP to drain the pool.

Recommendation: Use TWAP oracle (Chainlink or Uniswap V3 with 15-minute minimum window). Alternatively adopt minimum of (amount/reserve) * totalSupply approach.

Severity: HIGH

H-08 | UPGRADER_ROLE Can Drain All Protocol Funds Instantly — No Timelock

File: contracts/Factory.sol | **Lines:** 568-582

UPGRADER_ROLE can replace router via setRouter() and bsktUtils via setBSKTUtils() with no delay. Attack: replace bsktUtils with contract returning verifySignature() = true always; replace router with malicious contract accepting ETH but returning 1 wei output. All baskets drain in one block. Blast radius is 100% of protocol TVL. Also: monthlyFeeRate in initialize() has no bounds — setting to 1e18 causes immediate division by zero in calMgmtFee(), bricking all basket mint/burn operations.

Recommendation: Implement 48-72 hour TimelockController on setRouter(), setBSKTUtils(), and all implementation updates. Add require(_monthlyFeeRate > 0 && _monthlyFeeRate <= 1e16) in initialize().

Severity: HIGH

H-09 | Signature Replay — No Chain ID, No msg.sender, No Nonce

File: contracts/Utils/BSKTUtils.sol | **Lines:** 70-71

messageHash = keccak256(abi.encode(deadline, tokens, weights, data)) omits block.chainid, address(this), msg.sender, and nonce. A valid signature is replayable across every EVM chain, usable by any observer before deadline, and reusable multiple times. Front-running createBSKT() with the same signature makes the attacker the permanent Manager NFT holder. All 5 call sites are affected: createBSKT(), contribute(), withdrawETH(), claimFee(), rebalance().

Recommendation: Include block.chainid, address(this), msg.sender, and incrementing nonce in the hash. Adopt EIP-712 domain separation.

Severity: HIGH

H-10 | No On-Chain Minimum Output on Swap Operations

File: contracts/BSKT.sol, contracts/Factory.sol | **Lines:** 469-480, 602-611, 1047-1054, 505

Every swap validates only amounts[i] > 0. One wei of output satisfies this check. Users have zero independent on-chain slippage control. The swap router is a generic changeable address — not hardcoded to any specific aggregator. Off-chain signed calldata is the only protection. Sandwich attacks can extract near-total user funds on any swap: contribute(), withdrawETH(), rebalance(), createBSKT().

Recommendation: Add uint256[] calldata _minAmountsOut to all swap functions. Enforce require(amounts[i] >= _minAmountsOut[i]) after each swap.

Severity: HIGH

H-11 | DEFAULT_ADMIN_ROLE on veALVA Can Burn Any User's Tokens — Permanent ALVA Lock

File: contracts/tokens/veALVA.sol | **Lines:** 20, 28-31

veALVA.initialize() only grants DEFAULT_ADMIN_ROLE to deployer, not ADMIN_ROLE. DEFAULT_ADMIN_ROLE can grant ADMIN_ROLE to themselves at any time. burnTokens(victim, 1 wei) reduces victim's veALVA below lockData.votingPower. When victim calls unstake(), veALVA.burnTokens(victim, votingPower) panics on insufficient balance. Victim can never unstake. Their ALVA is permanently locked. Attack cost: one transaction.

Recommendation: Deployer must renounceRole(DEFAULT_ADMIN_ROLE) after granting ADMIN_ROLE to staking contract. In unstake(), use Math.min(lockData.votingPower, veALVA.balanceOf(msg.sender)) for the burn amount.

Severity: HIGH

H-12 | BURN_ROLE Can Burn Any User's Tokens Without Approval (AlvaraAvax)

File: contracts/tokens/AlvaraAvax.sol | **Lines:** 29-31

burn(address from, uint256 amount) calls super._burn() directly bypassing ERC20 allowance checks. Unlike standard burnFrom() which requires approved allowance, BURN_ROLE can destroy any user's ALVA-Avax balance unconditionally. All roles (MINTER, BURN, DEFAULT_ADMIN) are granted to the same deployer address.

Recommendation: Replace `super._burn()` with `_spendAllowance()` + `super._burn()` to enforce standard ERC20 approval. Restrict `BURN_ROLE` to bridge contract only.

Severity: HIGH

H-13 | GREYLIST_MANAGER_ROLE Traps All Greylisted User Funds Permanently

File: `contracts/tokens/Alvara.sol` | **Lines:** 74-79

`addToGreyList()` has no timelock and no notice period. A greylisted user cannot transfer ALVA to anyone, cannot stake, cannot interact with any protocol function. Their ALVA is trapped permanently until the role holder removes them. The same `GREYLIST_MANAGER_ROLE` that protects the protocol can weaponize it against individual users.

Recommendation: Add minimum 24-hour pending greylisting before activation. Provide emergency exit window. Allow transfer to designated recovery address for greylisted users.

Severity: HIGH

H-14 | veALVA Inflation via renewStaking to Lower-Ratio Pool

File: `contracts/AlvaStakingV3.sol` | **Lines:** 491-493

`_stake()` mints veALVA when `votingPowerTotal > votingPowerOld` but never burns when `votingPowerTotal < votingPowerOld`. Renewing from Pool A (ratio=2x) to Pool B (ratio=1x, same duration): user keeps 200 veALVA but correct value is 100. Phantom 100 veALVA persists permanently since `unstake()` only burns `lockData.votingPower` which records the new value.

Recommendation: Add else if (`votingPowerOld > votingPowerTotal`) `veALVA.burnTokens(msg.sender, votingPowerOld - votingPowerTotal)`.

Severity: HIGH

H-15 | ALVA Greylist DoS Bricks All AlvaStakingV3 Unstakes and Claims

File: `contracts/AlvaStakingV3.sol`, `contracts/tokens/Alvara.sol` | **Lines:** 351, 367, 111

If `GREYLIST_MANAGER_ROLE` greylists the `AlvaStakingV3` contract address, all `ALVA.transfer()` FROM the staking contract revert: `unstake()` fails, `claimRewards()` fails. Every user's staked ALVA and pending rewards are permanently inaccessible in one transaction.

Recommendation: Add `require(!ALVA.isGreyListed(address(this)))` guard in `unstake()` and `claimRewards()`. Implement governance restriction preventing `GREYLIST_MANAGER_ROLE` from targeting system contracts.

Severity: HIGH

H-16 | ADMIN_ROLE Can Drain Staking Pool via Reward State Manipulation

File: `contracts/AlvaStakingV3.sol` | **Lines:** 1007-1071

Three `ADMIN_ROLE` functions — `updateAllIncrementedValues()`, `updateOpeningRewardIncrement()`, `addMissedRewards()` — can arbitrarily rewrite reward accounting for any lock. Compromised admin can set own `accountTotalCalculatedRewards` to full contract ALVA balance then `claimRewards()` to drain entire pool.

Recommendation: Require multi-sig. Add cumulative caps. Implement 24-hour timelocks on `updateAllIncrementedValues()` and `updateOpeningRewardIncrement()`. Emit detailed events for every change.

Severity: HIGH

H-17 | Dual Lock via isActive Late Write — Concrete Reentrancy Path

File: `contracts/AlvaStakingV3.sol` | **Lines:** 502-532

Between `accountToLockIds.push(N)` at line 502 and `isActive=true` at line 532, lock N is in the array with `isActive=false`. During `ALVA.transferFrom()` at line 508, a callback can reenter `stake()`. The reentrant call finds `isActive=false` and creates lock N+1. Both locks finish with `isActive=true`, violating the one-lock invariant. Lock N's ALVA becomes permanently trapped.

Recommendation: Move `isActive=true` to before the external call at line 508, or add `nonReentrant` to all user-facing functions.

Severity: HIGH

H-18 | Instant Signer Replacement — No Timelock on updateSigner

File: `contracts/utils/BSKTUtils.sol` | **Lines:** 45-49

`updateSigner()` allows owner to instantly replace the signer with no delay. The signer controls all signature verification across 5 protocol entry points. A compromised owner key calls `updateSigner(attacker_signer)`, then attacker signs arbitrary calldata enabling unauthorized basket operations protocol-wide.

Recommendation: Implement two-step signer rotation with minimum 48-hour delay. Emit `SignerUpdated(oldSigner, newSigner)` event immediately for off-chain monitoring.

Severity: HIGH

8. MEDIUM Severity Findings

M-01 | addMissedRewards Overwrites Instead of Accumulates Pending Rewards

File: contracts/AlvaStakingV3.sol | Line: 1070

Uses = instead of += when setting accountTotalCalculatedRewards. Admin calling addMissedRewards(user, 100) when user has 500 ALVA pending results in balance = 100, permanently erasing 400 ALVA.

Recommendation: Change = to +=. Add event emission for every admin reward modification.

Severity: MEDIUM

M-02 | _calculateRewards Division by Zero — Zero Pool Locked

File: contracts/AlvaStakingV3.sol | Lines: 691-695

Division by poolToAmountLocked[pool] with no zero guard. If all stakers exit before topUpRewards() processes a period, any user who stakes after that call has openingRewardId pointing to the zero-locked period. claimRewards() permanently panics.

Recommendation: Add if (poolLocked == 0) return 0 before the division.

Severity: MEDIUM

M-03 | veAlvaBalance Division by Zero — decayInterval Initialized as Zero

File: contracts/AlvaStakingV3.sol | Lines: 941-943, 207

initialize() sets decayInterval = _decayInterval with no > 0 validation. If deployed with 0, veAlvaBalance() permanently panics on both divisions, bricking all governance and voting power.

Recommendation: Add require(_decayInterval > 0) in initialize().

Severity: MEDIUM

M-04 | Unbounded updateMinimumRewardAmount Enables Permanent Reward DoS

File: contracts/AlvaStakingV3.sol | Lines: 705-709, 415-418

No bounds check on minimumRewardAmount and no event. Setting to type(uint256).max permanently bricks topUpRewards(). Setting to 0 allows empty reward periods.

Recommendation: Add require(amount >= 1 && amount <= MAX_REWARD_THRESHOLD). Emit RewardAmountUpdated event.

Severity: MEDIUM

M-05 | Missing nonReentrant on All User-Facing Functions

File: contracts/AlvaStakingV3.sol | Lines: 239, 253, 286, 343, 365, 376

ReentrancyGuardUpgradeable not inherited. stake(), renewStaking(), unstake(), claimRewards(), increaseAmount(), compoundRewards() all make external calls (veALVA.mint, ALVA.transferFrom) after state changes. If ALVA or veALVA are upgraded to include hooks, reentrancy becomes exploitable.

Recommendation: Inherit ReentrancyGuardUpgradeable and add nonReentrant to all six functions.

Severity: MEDIUM

M-06 | poolToExpiredAmount Underflow Risk in renewStaking

File: contracts/AlvaStakingV3.sol | Line: 313

Subtracts previousAmount from poolToExpiredAmount with no safety check. If accounting inconsistency makes the value less than previousAmount, Solidity 0.8 panics permanently, blocking user from ever calling renewStaking() again.

Recommendation: Add require check before subtraction: require(poolToExpiredAmount[closingId][pool] >= previousAmount).

Severity: MEDIUM

M-07 | Compound Attack — Free Timer Reset Combined With veALVA Inflation

File: contracts/AlvaStakingV3.sol | **Lines:** 286-337, 491-493

Combining H-14 (inflation) and free lock reset: stake 100 ALVA in Pool A (2x ratio) → get 200 veALVA → renewStaking(0, PoolB[1x]) → retain 100 phantom veALVA at zero cost → repeat each cycle. Over N cycles: N*100 phantom veALVA with only 100 ALVA locked, permanently distorting governance.

Recommendation: Fix the missing burn in _stake(). Require amount > 0 for pool-switching renewals.

Severity: MEDIUM

M-08 | increaseAmount on Expiring Lock Enables Flash Governance Power

File: contracts/AlvaStakingV3.sol | **Lines:** 253-271

increaseAmount() only checks endTime > block.timestamp with no minimum remaining duration. User adds large ALVA to a lock expiring in 1 second, gets instant proportional veALVA. If governance vote is active at that moment, user holds massively inflated power for 1 second. Then unstakes recovering all capital.

Recommendation: Add require(lockIdToLockData[lockId].endTime >= block.timestamp + MIN_REMAINING_DURATION) where MIN_REMAINING_DURATION >= REWARD_PERIOD.

Severity: MEDIUM

M-09 | ADMIN_ROLE Not Granted in veALVA.initialize — Staking Deployment Fragility

File: contracts/tokens/veALVA.sol | **Lines:** 15-21

initialize() only grants DEFAULT_ADMIN_ROLE. ADMIN_ROLE (required for mint and burnTokens) must be separately granted. If deployment script omits grantRole(ADMIN_ROLE, stakingContract), no staking functions work. stake() and unstake() permanently revert.

Recommendation: Add staking contract address as parameter in veALVA.initialize() and grant ADMIN_ROLE immediately.

Severity: MEDIUM

M-10 | burnTokens Callable on Any Account Without Approval

File: contracts/tokens/veALVA.sol | **Lines:** 28-31

burnTokens(address account, uint256 amount) calls _burn() directly with no allowance check. Unlike ERC20 burnFrom() which requires allowance, any ADMIN_ROLE holder can destroy any user's veALVA unconditionally.

Recommendation: Restrict burnTokens to only the designated staking contract address rather than any ADMIN_ROLE holder.

Severity: MEDIUM

M-11 | MINTER_ROLE Has No Supply Cap on AlvaraAvax

File: contracts/tokens/AlvaraAvax.sol | **Lines:** 25-27

mint() has no maximum supply validation. MINTER_ROLE can mint type(uint256).max ALVA-Avax, destroying token value and diluting all holders. No cross-chain supply coordination exists on-chain.

Recommendation: Define MAX_SUPPLY equal to expected cross-chain allocation. Add require(totalSupply() + amount <= MAX_SUPPLY) in mint().

Severity: MEDIUM

M-12 | Admin Key Loss Permanently Disables DEX Trading (AlvaraAvax)

File: contracts/utils/WithSupervisedTransfersAvax.sol | **Lines:** 41-56

When listingTimestamp = 0 (default), duringSupervisedTransfers is permanently true, blocking all transferFrom calls not involving admin or whitelist. If deployer loses key before setListingTimestamp(), token is permanently stuck — DEX trading impossible forever.

Recommendation: Add maximum supervision period after which supervised transfers end automatically. Transfer DEFAULT_ADMIN_ROLE to multi-sig immediately after deployment.

Severity: MEDIUM

M-13 | All Critical Roles Granted to Same Deployer on AlvaraAvax

File: contracts/tokens/AlvaraAvax.sol | **Lines:** 64-68

DEFAULT_ADMIN_ROLE, MINTER_ROLE, and BURN_ROLE all granted to same msg.sender. Single key compromise = unlimited minting, arbitrary burns from any user, listing timestamp control.

Recommendation: Transfer each role to a dedicated multi-sig. Grant MINTER_ROLE and BURN_ROLE only to the bridge contract.

Severity: MEDIUM

M-14 | Greylisted User Funds Permanently Trapped — No Recovery Path

File: contracts/tokens/Alvara.sol | **Lines:** 106-114

Greylisted user cannot transfer ALVA to anyone, cannot stake, cannot interact with any ALVA-requiring protocol function. No timelock on greylisting, no emergency exit, no recovery mechanism.

Recommendation: Add minimum 24-hour notice before greylist activates. Allow transfer to approved recovery address for greylisted users.

Severity: MEDIUM

M-15 | DEFAULT_ADMIN No Timelock on Greylist Management

File: contracts/tokens/Alvara.sol | **Lines:** 74, 86

DEFAULT_ADMIN_ROLE can instantly grant GREYLIST_MANAGER_ROLE. GREYLIST_MANAGER_ROLE can instantly greylist any address. Compromised DEFAULT_ADMIN key can greylist all protocol contracts in one block.

Recommendation: Implement TimelockController for DEFAULT_ADMIN_ROLE operations. Require multi-sig for GREYLIST_MANAGER_ROLE.

Severity: MEDIUM

M-16 | Admin Key Loss Permanently Disables DEX Trading (Alvara.sol)

File: contracts/utils/WithSupervisedTransfers.sol | **Lines:** 41-56

Same issue as M-12 but for Ethereum ALVA. If deployer loses key before setListingTimestamp(), DEX trading via transferFrom is permanently blocked.

Recommendation: Same as M-12 — maximum supervision period and multi-sig ownership.

Severity: MEDIUM

M-17 | 200 Million ALVA Minted to Single Deployer Address

File: contracts/tokens/Alvara.sol | **Line:** 67

Entire 200M ALVA supply minted to msg.sender in one transaction. Deployer temporarily holds 100% of supply. Key compromise between deployment and distribution loses the entire token supply.

Recommendation: Use a time-locked distribution contract as the recipient rather than a raw EOA.

Severity: MEDIUM

M-18 | Missing ecrecover address(0) Check — Signature Bypass on Misconfiguration

File: contracts/utils/BSKTUtils.sol | **Lines:** 67-84, 37-40

verifySignature() compares ecrecover result against signer without checking for address(0). initialize() sets signer without zero validation (updateSigner has the check). If initialized with signer=address(0), any malformed signature passes verification, bypassing all protocol signature protection.

Recommendation: Add if (recoveredSigner == address(0)) return false. Add if (_signer == address(0)) revert InvalidSigner() in initialize().

Severity: MEDIUM

M-19 | Signature Malleability — No s-Value Validation

File: contracts/utils/BSKTUtils.sol | **Lines:** 93-118

splitSignature() does not validate s is in the lower half of secp256k1 curve order. For any valid signature (r,s,v), an equivalent (r, n-s, v') exists recovering to the same signer, violating EIP-2.

Recommendation: Add s-value bounds check or replace custom implementation with OpenZeppelin's ECDSA.recover().

Severity: MEDIUM

M-20 | feeCollector DoS — Push Payment Blocks Basket Creation and Withdrawals

File: contracts/Factory.sol, contracts/BSKT.sol | **Lines:** 445, 433, 669

ETH fees pushed to feeCollector via .call. If feeCollector cannot receive ETH, every fee-deducting transaction fails globally — blocking basket creation, all contributions, and all ETH withdrawals simultaneously.

Recommendation: Replace push with pull: accumulate in pendingFees[feeCollector] mapping and add withdrawFees() function.

Severity: MEDIUM

M-21 | Retroactive Fee Hike Affects All Existing Basket Users

File: contracts/Factory.sol | **Lines:** 787-798

additionalPlatformFeeRate is a global Factory variable read live by every BSKT. Raising the rate from 0% to 5% instantly affects all existing baskets with no notice, no grace period, and no opt-out.

Recommendation: Store rate per-basket at creation time as immutable. Or implement 7-day timelock for rate changes.

Severity: MEDIUM

M-22 | bsktUtils Not Set in Factory.initialize — Protocol Dead on Deployment

File: contracts/Factory.sol | **Lines:** 328-387

bsktUtils defaults to address(0). Calls to IBSKTUtils(address(0)).verifySignature() decode empty bytes as false, making every createBSKT() permanently revert until setBSKTUtils() is called separately.

Recommendation: Add _bsktUtils as required parameter in initialize() with isContract() validation.

Severity: MEDIUM

M-23 | monthlyFeeRate Unvalidated — Division by Zero Bricks All Baskets

File: contracts/Factory.sol | **Lines:** 384, 727-742

monthlyFeeRate = _monthlyFeeRate with no bounds check. If set to 1e18: 1e18 - 1e18 = 0, powu(0, months) = 0, numerator/0 panics. calMgmtFee is called inside BSKTPair.mint() and burn() via distMgmtFee(), permanently bricking every basket.

Recommendation: Add require(_monthlyFeeRate > 0 && _monthlyFeeRate <= 1e16) in initialize().

Severity: MEDIUM

M-24 | updateTokens No Length Validation — _tokens vs _reserves Mismatch

File: contracts/tokens/BSKTPair.sol | **Lines:** 153-158

updateTokens() only validates _tokens.length != 0 but not _tokens.length == _reserves.length. Mismatched arrays permanently corrupt basket reserve accounting, bricking all withdrawals.

Recommendation: Add require(_tokens.length == _reserves.length, "Array length mismatch").

Severity: MEDIUM

M-25 | distMgmtFee Is Public — Anyone Can Spam Maximum Fee Extraction

File: contracts/tokens/BSKTPair.sol | **Line:** 247

distMgmtFee() has no access restriction. Any address can call it every 1800 seconds to ensure maximum fee accrual for the basket owner. Combined with the 1800s testnet constant, fully automated maximum extraction is trivially achievable.

Recommendation: Restrict to onlyOwner or make internal called only from mint() and burn(). If keeper bots are intended, add minimum time gap enforcement.

Severity: MEDIUM

M-26 | Single Blacklisted Token Permanently Bricks All Basket Withdrawals

File: contracts/tokens/BSKTPair.sol, contracts/BSKT.sol | **Lines:** 210-222

burn() uses safeTransfer in a loop over all basket tokens. One non-transferable token (USDC blacklists BSKTPair, token pauses) reverts entire burn(), blocking all user withdrawals permanently.

Recommendation: Implement per-token try-catch or add withdrawSpecificTokens() function allowing selective token withdrawal.

Severity: MEDIUM

M-27 | CEI Violation and Missing Reentrancy Guard in AlvaraDao

File: contracts/AlvaraDao.sol | Lines: 45-75

enter() mints xALVA before ALVA.transferFrom(). leave() calls xALVA.burnFrom() before ALVA.transfer(). Neither function has nonReentrant. If tokens are upgraded with hooks, reentrancy becomes exploitable.

Recommendation: Add ReentrancyGuardUpgradeable and nonReentrant to both functions. Reorder enter() to transfer ALVA in before minting shares.

Severity: MEDIUM

M-28 | Blacklist Permanently Locks User Funds in AlvaraDao

File: contracts/AlvaraDao.sol | Lines: 46, 63, 83-88

Both enter() and leave() check blacklistAddress[msg.sender]. Blacklisted user with xALVA has no withdrawal path — no admin recovery function, no emergency exit, no timelock on blacklisting.

Recommendation: Add emergencyWithdraw(address blacklistedUser, address recovery) onlyOwner. Add 48-hour timelock on blacklistUser().

Severity: MEDIUM

M-29 | Quadratic Rebalance Gas — $O(N \times M)$ Makes Large Baskets Permanently Stuck

File: contracts/BSKT.sol | Lines: 1024-1070

_swapTokens() nested loops: N old tokens $\times$ M new tokens = $N \times M$ swap operations. For 15 $\times$ 15 tokens: 225 operations $\approx$ 28M gas, approaching 30M block limit. Baskets with >17 tokens per side become permanently un-rebalanceable.

Recommendation: Refactor to linear 2N pattern using WETH as intermediate. Add MAX_TOKENS = 20 in _checkValidTokensAndWeights().

Severity: MEDIUM

M-30 | Strict Equality in _tokensToEth Blocks Exotic Token Withdrawals

File: contracts/BSKT.sol | Line: 604

tokenBalanceAfter != (tokenBalanceBefore - _amounts[i]) strict equality. For rebasing tokens or deflationary sender-burn tokens, this reverts. Since _tokensToEth is called by withdrawETH() and claimFee(), any basket containing such tokens permanently bricks its ETH exit path.

Recommendation: Replace with directional check: if (deltaEth == 0 || tokenBalanceAfter > tokenBalanceBefore - _amounts[i]) revert.

Severity: MEDIUM

M-31 | feeCollector Push Payment Blocks Contributions and ETH Withdrawals (BSKT)

File: contracts/BSKT.sol | Lines: 433, 669

Same pattern as M-20 in BSKT. _sendETH to feeCollector fails if collector cannot receive ETH, blocking all contribute() and withdrawETH() globally.

Recommendation: Apply pull payment pattern as per M-20 recommendation.

Severity: MEDIUM

M-32 | Retroactive Fee Hike in BSKT — Live Rate Impacts Existing Users

File: contracts/BSKT.sol | Lines: 424-428, 525-527, 659-663

contribute(), withdraw(), and withdrawETH() all read additionalPlatformFeeRate live from Factory. Same issue as M-21 but manifesting in all three user-facing BSKT functions simultaneously.

Recommendation: Same as M-21 — snapshot per-basket at creation or implement timelock.

Severity: MEDIUM

9. LOW Severity Findings (Summary)

#	ID	Title	File	Lines
1	L-01	Missing __gap in Factory.sol	Factory.sol	Various
2	L-02	setRouter Missing isContract Check	Factory.sol	578-582
3	L-03	Unbounded setMinLpWithdrawal — No Min or Max	Factory.sol	400-403
4	L-04	Double Tax on Fee-on-Transfer Tokens	Factory.sol	496-499
5	L-05	Fee Calculated Before Weights Validated	Factory.sol	441 vs 467
6	L-06	Missing __gap in BSKT.sol	BSKT.sol	Various
7	L-07	Missing disableInitializers in BSKT.sol	BSKT.sol	—
8	L-08	isContractAddress False During Construction	BSKT.sol	1135-1137
9	L-09	Owner Malicious Rebalance to Worthless Token	BSKT.sol	728, 713
10	L-10	burn() CEI Violation — reserves After Transfer	BSKTPair.sol	210-222
11	L-11	First Depositor Fixed 1000 LP — No Dead Shares	BSKTPair.sol	178
12	L-12	Rounding in _totalReservedETH Favors Contributors	BSKTPair.sol	453
13	L-13	mint() No amounts.length Validation	BSKTPair.sol	169-188
14	L-14	Missing disableInitializers in BSKTPair.sol	BSKTPair.sol	—
15	L-15	Missing __gap in BSKTPair.sol	BSKTPair.sol	Various
16	L-16	Orphaned Funds After Rebalance	BSKTPair.sol	130-146
17	L-17	deductClaimedFee No Guard	BSKTPair.sol	235-239
18	L-18	calculateShareLP 1000 ether Fallback Risk	BSKTPair.sol	283
19	L-19	Missing __gap in AlvaStakingV3.sol	AlvaStakingV3.sol	Various
20	L-20	Forever Lock Phantom Expired Entries	AlvaStakingV3.sol	576-584
21	L-21	getTotalLocks Returns Swapped Values	AlvaStakingV3.sol	992-996
22	L-22	Raw transfer Without SafeERC20 (Staking)	AlvaStakingV3.sol	351, 367, 508
23	L-23	burnFrom Return Value Not Checked	AlvaStakingV3.sol	520
24	L-24	Reward Discrepancy on increaseAmount	AlvaStakingV3.sol	587
25	L-25	Pool Status After State Changes in renewStaking	AlvaStakingV3.sol	296-307
26	L-26	minimumStakingAmount Not in renewStaking	AlvaStakingV3.sol	286-289
27	L-27	pop() Fragile Assumption in renewStaking	AlvaStakingV3.sol	323
28	L-28	Rewards Not Finalized Before Lock Closure	AlvaStakingV3.sol	305-307
29	L-29	Unbounded accountToLockIds Array Growth	AlvaStakingV3.sol	502
30	L-30	Free Timer Reset via renewStaking(0, pool)	AlvaStakingV3.sol	286-337
31	L-31	Zero veALVA Minting on Dust Stakes	AlvaStakingV3.sol	491, 915-919
32	L-32	duration vs rewardPeriods Misalignment	AlvaStakingV3.sol	488, 528-530
33	L-33	isActive Late Write in FOREVER Lock Path	AlvaStakingV3.sol	518-532

#	ID	Title	File	Lines
34	L-34	No Maximum veAlvaRatio Validation	AlvaStakingV3.sol	223
35	L-35	Short-Duration Pool Exploit	AlvaStakingV3.sol	488, 528-530
36	L-36	Missing __gap in AlvaraDao.sol	AlvaraDao.sol	Various
37	L-37	Raw transfer Without SafeERC20 (DAO)	AlvaraDao.sol	58, 72
38	L-38	lockedPeriod Not Validated in initialize	AlvaraDao.sol	41
39	L-39	Missing __gap in BSKTUtils.sol	BSKTUtils.sol	—
40	L-40	Missing __gap in veALVA.sol	veALVA.sol	—
41	L-41	approve() Can Be Called on Non-Transferable Token	veALVA.sol	33-36
42	L-42	Missing disableInitializers in WETH.sol	WETH.sol	—
43	L-43	Missing __gap in WETH.sol	WETH.sol	—
44	L-44	Both withdraw() Functions Use .transfer()	WETH.sol	34, 49
45	L-45	Missing disableInitializers in AlvaraAvax.sol	AlvaraAvax.sol	—
46	L-46	setListingTimestamp Accepts Past Timestamps (Avax)	WithSupervisedTransfersAvax.sol	75-84
47	L-47	listingTimestamp Reset to Zero Before Listing (Avax)	WithSupervisedTransfersAvax.sol	75-84
48	L-48	Missing __gap in AlvaraAvax.sol	AlvaraAvax.sol	—
49	L-49	Missing __gap in Alvara.sol	Alvara.sol	—
50	L-50	setListingTimestamp Accepts Past Timestamps	WithSupervisedTransfers.sol	70-79
51	L-51	listingTimestamp Reset to Zero Before Listing	WithSupervisedTransfers.sol	70-79
52	L-52	Greylist Only Checks Sender — Asymmetric Restriction	Alvara.sol	106-114

10. INFORMATIONAL Findings (Summary)

#	Title	File	Lines
I-01	createBSKT Deploys Contracts Before Signature Verification	Factory.sol	457-467
I-02	Factory Has No receive() — Router ETH Refunds Revert	Factory.sol	—
I-03	ALVA Fee Gaming via Immediate Rebalance	Factory.sol	812-838
I-04	ALVA Fee Gaming via rebalance()	BSKT.sol	728
I-05	_tokensToEth Loop No Array Bound Check	BSKT.sol	586-588
I-06	ETH Balance Delta Measurement Per Iteration Risk	BSKT.sol	586-597
I-07	Repeated Storage Reads in renewStaking	AlvaStakingV3.sol	291-331
I-08	LockRenewed Event Missing Critical Fields	AlvaStakingV3.sol	336
I-09	Precision Loss in _calculateRewards	AlvaStakingV3.sol	695
I-10	No Event Emitted on Signer Update	BSKTUtils.sol	45-49
I-11	mint() and burnTokens() Declared public Instead of external	veALVA.sol	23, 28

#	Title	File	Lines
I-12	No Maximum Supply Cap on veALVA mint	veALVA.sol	23-26
I-13	withdraw(uint256) Allows Zero Amount	WETH.sol	46-51
I-14	initialize() Declared public Instead of external (WETH)	WETH.sol	17
I-15	initialize() Declared public Instead of external (AlvaraAvax)	AlvaraAvax.sol	18
I-16	No Initial Supply — Undocumented Bridge Architecture	AlvaraAvax.sol	18-23
I-17	initialize() Declared public Instead of external (Alvara)	Alvara.sol	62
I-18	GREYLIST_MANAGER_ROLE Not Granted in initialize	Alvara.sol	62-68
I-19	H-04/H-06 calMgmtFee and 1800s Period Are Directly Interdependent	BSKTPair.sol	259, 347
I-20	closingRewardId Edge Case Already Handled Correctly	AlvaStakingV3.sol	311
I-21	bsktImplementation Variable Name Misleading — Stores Beacon Address	Factory.sol	87
I-22	Floating Pragma Solidity Version	AlvaStakingV3.sol	1

11. General Recommendations

Priority 1 — Before Deployment:

1. Fix hardcoded 1800s → 30 days in BSKTPair (H-06)
2. Replace custom WETH.sol with canonical WETH9 (H-01)
3. Fix veALVA.initialize() to grant ADMIN_ROLE to staking contract and add deployer renouncement (H-11)
4. Fix xALVA inflation attack in AlvaraDao with dead shares (H-02)
5. Fix signature hash to include block.chainid, msg.sender (H-09)

Priority 2 — Before Mainnet Launch: 6. Implement TimelockController on Factory UPGRADER_ROLE and BSKTUtils owner (H-03, H-08, H-18) 7. Add ALVA greylist protection for system contracts (H-04, H-15) 8. Fix addMissedRewards to use += not = (M-01) 9. Add nonReentrant to all AlvaStakingV3 user functions (M-05) 10. Add on-chain minimum output parameters to all swap functions (H-10)

Priority 3 — Security Hardening: 11. Implement pull payment for fee collection (M-20, M-31) 12. Add per-basket immutable additionalPlatformFeeRate (M-21, M-32) 13. Fix veALVA burn in _stake() when moving to lower-ratio pool (H-14) 14. Add zero guard in _calculateRewards for zero pool locked (M-02) 15. Ensure all __gap arrays are added to all upgradeable contracts (L-01 through L-52)

12. Disclaimer

This report is provided for informational purposes only and represents the findings of a security audit conducted on the specified contracts at a specific point in time. This audit does not constitute an endorsement of the project or guarantee that the audited contracts are free from vulnerabilities. Security audits are not exhaustive and may not identify all potential issues. The auditors are not responsible for any losses arising from deployment or use of the audited contracts. Protocol teams are responsible for implementing recommended fixes and conducting additional testing before deployment.

End of Report